

Manual de Testes da API – Genesis v2.1.5

Data: 14 de Setembro de 2026

Versão da API: 2.1.5

URL Base (Produção): <https://garson.techconectar.com>

URL Base (Desenvolvimento): <http://localhost:3000>

Autenticação: Bearer Token (JWT) – obtido em </auth/login>

Swagger: <https://garson.techconectar.com/api/docs>

Credenciais de Teste

Papel	Email	Senha
ADMIN	admin@genesis.co.mz	admin123
FORNECEDOR	contacto@songogourmet.co.mz	senha123
CLIENTE	joao.carlos@genesis.co.mz	senha123
ESTAFETA	suamota.express@genesis.co.mz	senha123

| **Nota:** Substitui <TOKEN> pelo token obtido no login.

Índice de Módulos

1. Autenticação (/auth)
 2. Catálogo (/catalog)
 3. Carrinho (/carrinho)
 4. Pedidos (/orders)
 5. Pagamentos (/payments)
 6. Entregas (/deliveries)
 7. Administração (/admin)
 8. Anexos (/anexos)
 9. Avaliações (/avaliacoes)
 10. Cupões (/cupoes)
 11. Favoritos (/favoritos)
 12. Requisições Especiais (/requisicoes)
 13. Vendas Balcão (/vendas)
 14. Estabelecimentos (/estabelecimentos)
 15.  **Perfis** (/perfis)
 16.  **Pessoas** (/pessoas) — atualização de identidade
 17.  **Clientes** (/clientes) — limite de crédito
 18.  **Contactos** (/contactos) — leitura e escrita
 19.  **Chat** (/pedidos/:id/chat) — leitura e escrita
 20.  **Banners** (/banners) — via pessoa_anexo
-

1. Autenticação (/auth)

Método	Endpoint	Descrição	Auth
POST	/auth/login	Login do utilizador	✗
POST	/auth/register	Registo de novo cliente	✗
GET	/auth/me	Perfil do utilizador autenticado	✓

1.1. Login (ADMIN)

http

POST /auth/login

Content-Type: application/json

```
{  
  "email": "admin@genesis.co.mz",  
  "password": "admin123"  
}
```

1.2. Login (FORNECEDOR)

http

POST /auth/login

```
{  
  "email": "contacto@songogourmet.co.mz",  
  "password": "senha123"  
}
```

```
}
```

1.3. Login (CLIENTE)

http

POST /auth/login

```
{  
  "email": "joao.carlos@genesis.co.mz",  
  "password": "senha123"  
}
```

1.4. Login (ESTAFETA)

http

POST /auth/login

```
{  
  "email": "suamota.express@genesis.co.mz",  
  "password": "senha123"  
}
```

1.5. Registo de Cliente

http

POST /auth/register

Content-Type: application/json

```
{  
  "email": "novo.cliente@email.com",  
  "password": "senha123",  
  "nome_completo": "Novo Cliente",  
  "telefone": "+258841234567",  
  "endereco": "Bairro 1, Songo"  
}
```

1.6. Obter Perfil

http

GET /auth/me

Authorization: Bearer <TOKEN>

✅ **Correção v2.1.4:** O JWT inclui todas as permissões herdadas dos papéis (papeis.permissoes é carregado no AuthService.validateUser).

Log de rastreamento no arranque: [AuthService] Login: user=X papeis=[Y] permissoes=Z.

2. Catálogo (/catalog)

Método	Endpoint	Auth
GET	/catalog/produtos	×
GET	/catalog/produtos/:id	×

Método	Endpoint	Auth
GET	/catalog/categorias	✗
GET	/catalog/categorias/:id	✗
GET	/catalog/produtos/categoria/:categoriaId	✗
GET	/catalog/fornecedores	✗
GET	/catalog/fornecedor/:id/geolocalizacao	✗
POST	/catalog/produtos	✓ produtos:criar
PATCH	/catalog/produtos/:id	✓ produtos:editar
DELETE	/catalog/produtos/:id	✓ produtos:eliminar
POST	/catalog/categorias	✓ produtos:criar
PATCH	/catalog/categorias/:id	✓ produtos:editar
DELETE	/catalog/categorias/:id	✓ produtos:eliminar

2.1. Listar Produtos (Público)

http

GET /catalog/produtos?page=1&limit=5&search=hamburguer&categoria=4&minPreco=100&maxPreco=500

2.2. Listar Fornecedores

http

GET /catalog/fornecedores

Response (200 OK):

```
json
[
  {
    "idpessoa": 1,
    "nome_completo": "Songo Gourmet & Grill",
    "alcunha": "SongoGourmet",
    "slogan": "O melhor sabor da vila do Songo",
    "selo_verificacao": 1
  }
]
```

2.3. Geolocalização do Fornecedor

http

GET /catalog/fornecedor/1/geolocalizacao

Response (200 OK):

```
json
{
  "latitude": -15.6142,
  "longitude": 32.7719,
  "nome_completo": "Songo Gourmet & Grill"
}
```

|  Valores corretos: latitude < 0 (hemisfério sul) e longitude > 0 (leste).

2.4. Criar Produto (FORNECEDOR/ADMIN)

http

POST /catalog/produtos

Authorization: Bearer <TOKEN_FORNECEDOR>

```
{  
  "fornecedor_id": 1,  
  "categoria_id": 4,  
  "nome": "Novo Hambúrguer",  
  "descricao": "Hambúrguer com queijo e bacon.",  
  "preco": 350.00,  
  "preco_promocional": 300.00,  
  "stock": 20,  
  "status_produto": "DISPONIVEL"  
}
```

3. Carrinho (/carrinho)

Método	Endpoint	Descrição
GET	/carrinho	Obter carrinho atual
POST	/carrinho/itens	Adicionar item
PATCH	/carrinho/itens/:itemId	Atualizar quantidade
DELETE	/carrinho/itens/:itemId	Remover item

Método	Endpoint	Descrição
DELETE	/carrinho	Limpar carrinho
POST	/carrinho/finalizar	Finalizar carrinho

3.1. Obter Carrinho (corrigido)

http

GET /carrinho

Authorization: Bearer <TOKEN_CLIENTE>

Response (200 OK):

json

```
{  
  "id_carrinho": 15,  
  "usuario_id": 3,  
  "estado": "ABERTO",  
  "data_criacao": "2026-09-13T20:55:47.000Z",  
  "itens": []  
}
```

✓ **Correção v2.1.4:** O usuario_id agora é o idpessoa (não idusuario). Antes dava 500 por FK constraint.

3.2. Adicionar Item

http

POST /carrinho/itens

Authorization: Bearer <TOKEN_CLIENTE>

```
{  
  "produto_id": 1,  
  "quantidade": 2  
}
```

3.3. Finalizar Carrinho

http

POST /carrinho/finalizar

Authorization: Bearer <TOKEN_CLIENTE>

4. Pedidos (/orders)

Método	Endpoint	Descrição
POST	/orders	Criar pedido
GET	/orders	Listar pedidos (por papel)
GET	/orders/:id	Detalhe
PATCH	/orders/:id/status	Atualizar status (ADMIN)
PATCH	/orders/subpedido/:subId/status	Atualizar sub-pedido
POST	/orders/:id/cancelar	Cancelar
GET	/orders/fornecedor/:fornecedorId/subpedidos	Sub-pedidos por fornecedor

4.1. Criar Pedido

http

POST /orders

Authorization: Bearer <TOKEN_CLIENTE>

```
{
  "cliente_id": 3,
  "itens": [
    { "produto_id": 1, "quantidade": 2 },
    { "produto_id": 4, "quantidade": 3 }
  ],
  "endereco_entrega_alternativo": "Bairro 3, Songo",
  "taxa_entrega": 100,
  "metodo_pagamento": "MPESA"
}
```

4.2. Atualizar Sub-Pedido (FORNECEDOR)

http

PATCH /orders/subpedido/8/status

Authorization: Bearer <TOKEN_FORNECEDOR>

```
{ "status": "ACEITO_LOJA" }
```

4.3. Cancelar Pedido

http

POST /orders/4/cancelar

Authorization: Bearer <TOKEN>

```
{  
  "motivo_categoria": "DESISTENCIA_CLIENTE",  
  "descricao_detalhada": "Mudei de ideia"  
}
```

5. Pagamentos (/payments)

5.1. Registrar Pagamento

http

POST /payments

Authorization: Bearer <TOKEN>

```
{  
  "pedido_id": 4,  
  "metodo_pagamento": "MPESA",  
  "valor": 1090.00,  
  "referencia_transacao": "REF-123",  
  "numero_conta_pagador": "841234567"  
}
```

5.2. Webhook (Público)

http

POST /payments/webhook/mpesa

```
{  
  "TransactionID": "MPESA123",  
  "orderId": 4,  
  "amount": 1090.00,  
  "phoneNumber": "841234567"  
}
```

6. Entregas (/deliveries)

Método	Endpoint	Descrição
POST	/deliveries/pedido/:pedidoId	Criar entrega
PATCH	/deliveries/:id/assign	Atribuir estafeta
PATCH	/deliveries/:id/status	Atualizar status
POST	/deliveries/:id/confirmar	Confirmar
POST	/deliveries/tracking	Enviar localização
GET	/deliveries/:id/tracking	Histórico de rastreio
GET	/deliveries? fornecedor_id=X	Listar (com filtro) 

Método	Endpoint	Descrição
GET	/deliveries/:id	Detalhe
GET	/deliveries/pedido/:pedidoId	Entrega por pedido

6.1. Criar Entrega

http

POST /deliveries/pedido/4

Authorization: Bearer <TOKEN>

6.2. Listar Entregas com Filtro (v2.1.4)

http

GET /deliveries?fornecedor_id=1

Authorization: Bearer <TOKEN_ADMIN>

|  **Novo:** filtra apenas entregas cujo pedido contém sub-pedidos desse fornecedor.

6.3. Atribuir Estafeta

http

PATCH /deliveries/5/assign

Authorization: Bearer <TOKEN_ADMIN>

```
{ "estafeta_id": 2 }
```

6.4. Enviar Localização (ESTAFETA)

http

POST /deliveries/tracking

Authorization: Bearer <TOKEN_ESTAFETA>

```
{  
  "entrega_id": 2,  
  "latitude": -15.615100,  
  "longitude": 32.772500  
}
```

6.5. Confirmar Entrega

http

POST /deliveries/2/confirmar

Authorization: Bearer <TOKEN_ESTAFETA>

```
{ "codigo": "609354" }
```

7. Administração (/admin)

7.1. Resumo de Vendas

http

GET /admin/reports/sales?start=2026-08-01&end=2026-08-27

Authorization: Bearer <TOKEN_ADMIN>

7.2. Desempenho de Estafetas (corrigido)

http

GET /admin/reports/delivery-performance

Authorization: Bearer <TOKEN_ADMIN>

Response (200 OK):

json

```
[  
  { "estafeta_id": 2, "nome": "Sua Mota Express", "tempo_medio_min": 25.5 }  
]
```

7.3. Listar Utilizadores (sem password_hash)

http

GET /admin/usuarios?page=1&limit=10&tipo=FORNECEDOR&is_ativo=true

Authorization: Bearer <TOKEN_ADMIN>

7.4. Detalhe do Utilizador (sem password_hash)

http

GET /admin/usuarios/101

Authorization: Bearer <TOKEN_ADMIN>

|  **Correção v2.1.3:** password_hash **não** está presente.

7.5. Atualizar Papéis

http

PATCH /admin/usuarios/103/papeis

Authorization: Bearer <TOKEN_ADMIN>

```
{ "papeis": ["FORNECEDOR", "CLIENTE"] }
```

7.6. Ativar/Desativar Utilizador

http

PATCH /admin/usuarios/103/status

Authorization: Bearer <TOKEN_ADMIN>

```
{ "is_ativo": false }
```

8. Anexos (/anexos)

Método	Endpoint	Auth
POST	/anexos/upload	 JWT + anexos:upload
GET	/anexos	 Público
GET	/anexos/produto/:produtoId	 Público
GET	/anexos/pessoa/:pessoaId	 Público

Método	Endpoint	Auth
GET	/anexos/:id	 Público
DELETE	/anexos/:id	 JWT
DELETE	/anexos/:id/hard	 JWT

8.1. Registrar Metadados (id opcional)

http

POST /anexos/upload

Authorization: Bearer <TOKEN>

```
{
  "idpessoa": 1,
  "tipo_anexo": "produto",
  "nome_ficheiro_original": "foto.jpg",
  "caminho_armazenamento": "https://kugula.techconectar.com/upload_server1.0.1.php?file=...",
  "tamanho_bytes": 24576,
  "hash_sha256": "3f7c8b9a1e2d3c4b5a6f7e8d9c0b1a2f3e4d5c6b7a8f9e0d1c2b3a4f5e6d7c8b9a",
  "mime_type": "image/jpeg"
}
```

8.2. Listar Anexos de um Produto (público)

http

GET /anexos/produto/14

Response (200 OK):

json

```
[
  {
    "id": 1,
    "idpessoa": 1,
    "tipo_anexo": "produto",
    "nome_ficheiro_original": "lacoste_polo.jpg",
    "caminho_armazenamento": "https://kugula.techconectar.com/upload_server1.0.1.php?file=...",
    "tamanho_bytes": "41720",
    "hash_sha256": "...",
    "mime_type": "image/jpeg",
    "url": "https://kugula.techconectar.com/upload_server1.0.1.php?file=..."
  }
]
```



O campo url está sempre presente (fallback para caminho_armazenamento).

8.3. Listar Anexos de uma Pessoa (público) — corrigido v2.1.5

http

GET /anexos/pessoa/3

Response (200 OK):

json

```
[
  {
    "id": 1,
```

```
"idpessoa": 3,  
"caminho_armazenamento": "https://...",  
"url": "https://..."  
}  
]
```

✓ **Correção v2.1.5:** antes dava 500 por incompatibilidade TypeORM com a coluna config. Agora usa SQL puro.

8.4. Associar Anexo a Produto com Novo Tipo

http

POST /produto-anexos/produto/14

Authorization: Bearer <TOKEN_FORNECEDOR>

```
{  
  "anexo_id": 1,  
  "tipo": "estilo_vida",  
  "ordem": 1  
}
```

Tipos válidos: capa, frontal, traseira, lateral_esquerda, lateral_direita, cima, baixo, estilo_vida, dimensoes, variacao, embalagem, status.

8.5. Associar Anexo a Pessoa com Novo Tipo

http

POST /pessoa-anexos/pessoa/1

Authorization: Bearer <TOKEN>

```
{
```

```
"anexo_id": 5,  
"tipo": "avatar"  
}
```

Tipos válidos: avatar, capa_perfil, documento_id, thumbnail, historico, foto, outra_imagem, banner_hero, banner_secundario, banner_masonry, banner_evento.

8.6. Soft Delete

http

DELETE /anexos/1

Authorization: Bearer <TOKEN>

8.7. Hard Delete

http

DELETE /anexos/1/hard

Authorization: Bearer <TOKEN>

9. Avaliações (/avaliacoes)

9.1. Criar Avaliação (CLIENTE)

http

POST /avaliacoes

Authorization: Bearer <TOKEN_CLIENTE>

```
{
  "pedido_id": 1,
  "estabelecimento_id": 1,
  "produto_id": 1,
  "estafeta_id": 2,
  "classificacao": 5,
  "comentario": "Excelente serviço!"
}
```

Response (201 Created):

json

```
{
  "id_avaliacao": 19,
  "pedido_id": 1,
  "cliente_id": 3,
  "classificacao": 5,
  "comentario": "Excelente serviço!",
  "criado_em": "2026-08-27T10:00:00.000Z"
}
```

|  **Correção v2.1.2:** pedido genuinamente próprio → 201 (antes dava 403).

9.2. Listar Avaliações

http

GET /avaliacoes

Authorization: Bearer <TOKEN>

10. Cupões (/cupoes)

Método	Endpoint	Auth
POST	/cupoes	 ADMIN
GET	/cupoes	 ADMIN
GET	/cupoes/meus-usos	 cupoes:visualizar_proprios
POST	/cupoes/validar	
GET	/cupoes/:id	 ADMIN

10.1. Validar Cupão (Público)

http

POST /cupoes/validar

Content-Type: application/json

```
{  
  "codigo": "PROMO20",  
  "valorPedido": 200.00  
}
```

10.2. Histórico de Usos do Cliente (v2.1.4)

http

GET /cupoes/meus-usos

Authorization: Bearer <TOKEN_CLIENTE>

Response (200 OK):

json

```
[
  {
    "pedido_id": 5,
    "cupom_id": 3,
    "codigo": "PROMO20",
    "descricao": "20% de desconto em toda a loja",
    "tipo_desconto": "PERCENTUAL",
    "valor_desconto": "20.00",
    "valor_desconto_aplicado": "10.00",
    "aplicado_em": "2026-09-01T10:00:00.000Z"
  }
]
```

✓ **Correção v2.1.4:** antes dava 403 por 2 motivos — (1) permissão cupoes:visualizar_proprios não atribuída ao CLIENTE; (2) rota @Get(':id') declarada antes de @Get('meus-usos'). Ambas corrigidas.

11. Favoritos (/favoritos)

11.1. Adicionar Favorito

http

POST /favoritos/produto/4

Authorization: Bearer <TOKEN_CLIENTE>

11.2. Verificar Favorito

http

GET /favoritos/produto/1/check

Authorization: Bearer <TOKEN_CLIENTE>

Response (200 OK):

json

```
{ "isFavorito": true }
```

12. Requisições Especiais (/requisicoes)

12.1. Criar Requisição – todos os campos opcionais

http

POST /requisicoes

Authorization: Bearer <TOKEN_CLIENTE>

Content-Type: application/json

```
{
  "designacao": "Pizza Vegetariana Grande",
  "especificacoes": "Massa fina, molho de tomate, queijo mozzarella...",
  "quantidade": 1,
  "contacto_whatsapp": "+258840001122"
}
```

Response (201 Created):

json

```
{
  "id_requisicao": 46,
  "cliente_id": 3,
  "fornecedor_id": null,
  "designacao": "Teste",
  "especificacoes": "...",
  "quantidade": 1,
  "contacto_whatsapp": "+258840001122",
  "preco_sugerido": null,
  "categoria_sugerida_id": 1,
  "status": "PENDENTE",
  "produto_id": 41,
  "produto": {
    "id_produto": 41,
    "nome": "Teste",
    "preco": "0.00",
```

```
"status_produto": "SOB_ENCOMENDA",
"categoria_id": 1
}
}
```

Matriz de testes — 4/4 (v2.1.5)

categoria_sugerida_id	preco_sugerido	Resultado
-----------------------	----------------	-----------

omitido	omitido	 201
---------	---------	---

omitido	presente (100)	 201
---------	----------------	---

presente (1)	omitido	 201
--------------	---------	---

presente (1)	presente (1)	 201
--------------	--------------	---

 **Correção v2.1.5 (FINAL):** a causa real era o valor preco a ser enviado como ""0"" (string com aspas literais). Corrigido para String(createDto.preco_sugerido ?? 0). **Bug resolvido após 3 rondas.**

12.2. Aprovar Requisição (FORNECEDOR/ADMIN)

http

PATCH /requisicoes/4

Authorization: Bearer <TOKEN_FORNECEDOR>

```
{
  "status": "APROVADO",
  "preco_aprovado": 450.00,
  "categoria_aprovada_id": 4
}
```

```
}
```

| **Nota:** fornecedor_id é auto-atribuído do token.

12.3. Rejeitar Requisição

http

PATCH /requisicoes/4

Authorization: Bearer <TOKEN_FORNECEDOR>

```
{
```

```
  "status": "REJEITADO",
```

```
  "motivo_rejeicao": "Produto indisponível no momento"
```

```
}
```

13. Vendas Balcão (/vendas)

13.1. Realizar Venda (FORNECEDOR)

http

POST /vendas/balcao

Authorization: Bearer <TOKEN_FORNECEDOR>

```
{
```

```
  "itens": [
```

```
    { "produto_id": 1, "quantidade": 2 },
```

```
{ "produto_id": 4, "quantidade": 3 }  
],  
"endereco_entrega": "Venda presencial",  
"observacoes": "Cliente pagou em dinheiro"  
}
```

Response (201 Created):

```
json  
  
{  
  "message": "Venda balcão realizada com sucesso",  
  "pedido_id": 17,  
  "total": 780  
}
```

| **Nota:** fornecedor_id **não** deve ser enviado no body.

13.2. Listar Vendas

```
http  
  
GET /vendas/balcao  
Authorization: Bearer <TOKEN_FORNECEDOR>
```

14. Estabelecimentos (/estabelecimentos)

Método	Endpoint	Auth
POST	/estabelecimentos	 admin:gestao_usuarios

Método	Endpoint	Auth
GET	/estabelecimentos	✓ admin:gestao_usuarios
GET	/estabelecimentos/:id	✓ admin:gestao_usuarios
PATCH	/estabelecimentos/:id	✓ admin:gestao_usuarios
DELETE	/estabelecimentos/:id	✓ admin:gestao_usuarios

14.1. Criar Estabelecimento (ADMIN)

http

POST /estabelecimentos

Authorization: Bearer <TOKEN_ADMIN>

```
{
  "pessoa_id": 5,
  "nome_comercial": "Loja do Zé",
  "nuit": "123456789",
  "raio_entrega_km": 5.0,
  "tempo_medio_preparo_min": 30,
  "is_aberto": true,
  "comissao_plataforma_pct": 10.0,
  "dados_pagamento_mpesa": "841234567"
}
```

Efeitos automáticos:

- Cria o estabelecimento.
- Atualiza pessoa.tipo_pessoa para 'FORNECEDOR'.

- Atribui o papel FORNECEDOR ao utilizador (se ainda não tiver).
-

15. Perfis (/perfis)

Método Endpoint Auth

GET /perfis/me  JWT

15.1. Obter Perfil Completo (v2.1.4)

http

GET /perfis/me

Authorization: Bearer <TOKEN_CLIENTE>

Response (200 OK):

json

```
{
  "idpessoa": 3,
  "nome_completo": "João Carlos Manuel",
  "alcunha": "JCarlos",
  "slogan": null,
  "selo_verificacao": 0,
  "limites": "0.00",
  "limite_credito": "0.00",
  "endereco_fisico": "Bairro 3, Songo",
  "ponto_referencia": "Atrás da HCB Central",
```

```

"tipo_pessoa": "CLIENTE",
"data_criacao": "2026-08-16T04:23:27.000Z",
"data_modificacao": "2026-08-16T04:23:27.000Z",
"usuario": {
  "idusuario": 103,
  "email": "joao.carlos@genesis.co.mz",
  "is_ativo": true,
  "ultimo_login": "2026-08-16T07:25:00.000Z",
  "criado_em": "2026-08-16T07:23:27.000Z",
  "papeis": [{ "idpapel": 4, "nome": "CLIENTE" }]
},
"estabelecimento": null,
"estafeta_perfil": null
}

```

 **Novo em v2.1.4:** Substitui `api_obter_cliente_meu_perfil()` da API antiga. Se for FORNECEDOR, inclui estabelecimento; se for ESTAFETA, inclui estafeta_perfil.

16. Pessoas (/pessoas)

Método	Endpoint	Descrição	Auth
GET	/pessoas/meu-perfil	Perfil da pessoa autenticada	 JWT
PATCH	/pessoas/:id	Atualizar dados de identidade 	 JWT (só própria ou ADMIN)

16.1. Atualizar Perfil Próprio (v2.1.5)

http

PATCH /pessoas/3

Authorization: Bearer <TOKEN_CLIENTE>

Content-Type: application/json

```
{
  "nome_completo": "João Carlos Manuel",
  "endereco_fisico": "Bairro 3, Songo",
  "ponto_referencia": "Atrás da HCB Central",
  "latitude": "-15.6142",
  "longitude": "32.7719"
}
```

Response (200 OK): devolve o perfil completo atualizado (mesma estrutura do GET /perfis/me).

Regras:

- Só o próprio utilizador (ou ADMIN) pode editar.
- Se latitude/longitude forem fornecidos, o sistema atualiza/cria a geolocalização automaticamente.

17. Clientes (/clientes)

Método	Endpoint	Auth
PATCH	/clientes/:id/limite-	 admin:gestao_usuarios

Método	Endpoint	Auth
	credito	

17.1. Atualizar Limite de Crédito (ADMIN)

http

PATCH /clientes/3/limite-credito

Authorization: Bearer <TOKEN_ADMIN>

Content-Type: application/json

```
{  
  "limite_credito": 5000.00  
}
```

Response (200 OK):

json

```
{  
  "idpessoa": 3,  
  "limite_credito": "5000.00",  
  "message": "Limite de crédito atualizado"  
}
```

18. Contactos (/contactos)

Método	Endpoint	Auth
GET	/contactos/pessoa/:idpessoa	 JWT
POST	/contactos	 JWT (só própria) 
PATCH	/contactos/:id	 JWT (só própria) 
DELETE	/contactos/:id	 JWT (só própria) 

18.1. Listar Contactos de uma Pessoa

http

GET /contactos/pessoa/3

Authorization: Bearer <TOKEN_CLIENTE>

Response (200 OK):

json

```
{
  "pessoa": {
    "idpessoa": 3,
    "nome_completo": "João Carlos Manuel",
    "tipo_pessoa": "CLIENTE"
  },
  "contactos": [
    { "idcontacto": 4, "tipo_contacto": "TELEFONE", "descricao": "+258840001122" },
    { "idcontacto": 5, "tipo_contacto": "WHATSAPP", "descricao": "+258840001122" },
```

```
{ "idcontato": 6, "tipo_contato": "EMAIL", "descricao": "joao.carlos@email.com" }  
]  
}
```

18.2. Criar Contato (v2.1.5)

http

POST /contactos

Authorization: Bearer <TOKEN_CLIENTE>

Content-Type: application/json

```
{  
  "idpessoa": 3,  
  "tipo_contato": "WHATSAPP",  
  "descricao": "+258840001999"  
}
```

Response (201 Created):

json

```
{  
  "idcontato": 8,  
  "idpessoa": 3,  
  "tipo_contato": "WHATSAPP",  
  "descricao": "+258840001999",  
  "data_criacao": "2026-09-14T06:12:08.000Z"  
}
```

18.3. Atualizar Contacto

http

PATCH /contactos/8

Authorization: Bearer <TOKEN_CLIENTE>

```
{  
  "descricao": "+258840002000"  
}
```

18.4. Remover Contacto

http

DELETE /contactos/8

Authorization: Bearer <TOKEN_CLIENTE>

19. Chat (/pedidos/:id/chat)

Método	Endpoint	Auth
--------	----------	------

GET	/pedidos/:id/chat	 JWT
-----	-------------------	---

POST	/pedidos/:id/chat	 JWT 
------	-------------------	---

19.1. Listar Mensagens

http

GET /pedidos/1/chat

Authorization: Bearer <TOKEN_CLIENTE>

Response (200 OK):

json

```
[]
```

19.2. Enviar Mensagem (v2.1.5)

http

POST /pedidos/1/chat

Authorization: Bearer <TOKEN_CLIENTE>

Content-Type: application/json

```
{  
  "mensagem": "Olá, quando chega o pedido?"  
}
```

Response (201 Created):

json

```
{  
  "id_mensagem": 2,  
  "pedido_id": 1,  
  "remetente_id": 3,
```

```
"tipo_remetente": "CLIENTE",
"mensagem": "Mensagem de teste v2.1.5",
"lida": 0,
"data_envio": "2026-09-14T06:12:10.000Z"
}
```

Regras de acesso:

- ADMIN vê tudo.
 - Cliente dono vê o seu.
 - Fornecedor com sub-pedido no pedido vê também.
-

20. Banners (/banners)

Arquitetura: Banners são um **tipo de anexo** (pessoa_anexo.tipo = banner_*) — não existe tabela própria.

Método	Endpoint	Auth
GET	/banners	 Público
GET	/banners? fornecedor_id=X	 Público
GET	/banners/fornecedor/:id	 Público
POST	/banners	 ADMIN
PATCH	/banners/:id	 ADMIN
DELETE	/banners/:id	 ADMIN

20.1. Listar Banners (Público) — corrigido v2.1.5

http

GET /banners

Response (200 OK):

json

```
[
  {
    "id": 1,
    "tipo": "banner_hero",
    "titulo": "Promoção de Verão 2026",
    "imagem_url": "https://kugula.techconectar.com/upload_server1.0.1.php?file=...",
    "link_url": "/produtos?categoria=4",
    "ordem": 1,
    "peso": 15,
    "fornecedor_id": 1,
    "idpessoa_dono": 1,
    "data_inicio": "2026-09-01T00:00:00Z",
    "data_fim": "2026-12-31T23:59:59Z",
    "is_ativo": true,
    "config": {
      "titulo": "Promoção de Verão 2026",
      "link_url": "/produtos?categoria=4",
      "peso": 15,
      "ordem": 1,
      "data_inicio": "2026-09-01T00:00:00Z",
      "data_fim": "2026-12-31T23:59:59Z"
    }
  },
]
```

```
"anexo": {  
  "id": 1,  
  "nome_ficheiro_original": "banner_verao.jpg",  
  "mime_type": "image/jpeg",  
  "tamanho_bytes": 41720,  
  "hash_sha256": "..."  
}  
}  
]
```

✓ **Correção v2.1.5:** antes dava 500. Agora usa SQL puro e é tolerante a schema inconsistente.

Filtro automático: banners expirados (data_fim < agora) são excluídos.

20.2. Criar Banner (ADMIN)

http

POST /banners

Authorization: Bearer <TOKEN_ADMIN>

Content-Type: application/json

```
{  
  "idpessoa": 1,  
  "anexo_id": 1,  
  "tipo": "banner_hero",  
  "titulo": "Promoção de Verão 2026",  
  "link_url": "/produtos?categoria=4",  
  "peso": 15,  
  "ordem": 1,  
  "data_inicio": "2026-09-01T00:00:00Z",  
  "data_fim": "2026-12-31T23:59:59Z"
```

```
}
```

Tipos válidos: banner_hero, banner_secundario, banner_masonry, banner_evento.

20.3. Atualizar Banner (merge parcial)

http

PATCH /banners/1

Authorization: Bearer <TOKEN_ADMIN>

```
{ "peso": 25 }
```

| **Nota:** PATCH faz **merge** — envia apenas o campo que queres alterar. As restantes chaves do config são preservadas.

20.4. Remover Banner

http

DELETE /banners/1

Authorization: Bearer <TOKEN_ADMIN>

Response (200 OK):

json

```
{ "message": "Banner removido com sucesso" }
```

Script de Verificação Consolidada — v2.1.5 (14 Testes)

Guarda como verify-v2.1.5.sh:

```
bash
```

```
#!/bin/bash
```

```
# Verificação pós-correções v2.1.5 (14/09/2026)
```

```
BASE="https://garson.techconectar.com"
```

```
PASS=0; FAIL=0
```

```
TOKEN_ADMIN=$(curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
-d '{"email":"admin@genesis.co.mz","password":"admin123"}' | jq -r '.access_token')
```

```
TOKEN_CLIENTE=$(curl -s -X POST $BASE/auth/login -H "Content-Type: application/json" \
-d '{"email":"joao.carlos@genesis.co.mz","password":"senha123"}' | jq -r '.access_token')
```

```
check() {
```

```
  local name="$1" expected="$2" actual="$3"
```

```
  if [ "$actual" = "$expected" ]; then
```

```
    echo "  $name → $actual"; ((PASS++))
```

```
  else
```

```
    echo "  $name → $actual (esperado $expected)"; ((FAIL++))
```

```
  fi
```

```
}
```

```
echo "==== [1] GET /anexos (público) ====="
```

```
check "GET /anexos" "200" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/anexos)"
```

```
echo "==== [2] POST /anexos/upload sem token ====="
```

```
check "POST /anexos/upload" "401" "$(curl -s -o /dev/null -w '%{http_code}' -X POST $BASE/anexos/upload -H 'Content-Type: application/json' -d '{}')
```

```
echo "==== [3] GET /anexos/pessoa/3 ==="
```

```
check "GET /anexos/pessoa/3" "200" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/anexos/pessoa/3 -H "Authorization: Bearer $TOKEN_CLIENTE")"
```

```
echo "==== [4] GET /anexos/produto/14 tem url ==="
```

```
URL=$(curl -s $BASE/anexos/produto/14 | jq -r '[0].url // "AUSENTE")
```

```
[ "$URL" != "AUSENTE" ] && [ "$URL" != "null" ] && echo "  url presente" && ((PASS++)) || echo "  url ausente" && ((FAIL++))
```

```
echo "==== [5] Geolocalização lat/lon ==="
```

```
R=$(curl -s $BASE/catalog/fornecedor/1/geolocalizacao)
```

```
LAT=$(echo $R | jq -r '.latitude'); LON=$(echo $R | jq -r '.longitude')
```

```
if (( $(echo "$LAT < 0 && $LON > 0" | bc -l) )); then
```

```
  echo "  lat=$LAT lon=$LON"; ((PASS++))
```

```
else
```

```
  echo "  lat=$LAT lon=$LON"; ((FAIL++))
```

```
fi
```

```
echo "==== [6-9] POST /requisicoes (4 combinações) ==="
```

```
for cat in "omitido" "presente"; do
```

```
  for preco in "omitido" "presente"; do
```

```
    BODY='{ "designacao": "Teste", "especificacoes": "...", "quantidade": 1, "contacto_whatsapp": "+258840001122"'
```

```
    [ "$cat" = "presente" ] && BODY="$BODY, \"categoria_sugerida_id\": 1"
```

```
    [ "$preco" = "presente" ] && BODY="$BODY, \"preco_sugerido\": 100"
```

```
    BODY="$BODY}"
```

```
    CODE=$(curl -s -o /dev/null -w '%{http_code}' -X POST $BASE/requisicoes \
```

```
    -H "Authorization: Bearer $TOKEN_CLIENTE" -H "Content-Type: application/json" -d "$BODY")
```

```
[ "$CODE" = "201" ] && echo "  cat=$cat preco=$preco → 201" && ((PASS++)) || echo "  cat=$cat preco=$preco → $CODE"
&& ((FAIL++))
```

```
done
done
```

```
echo "==== [10] GET /cupoes/meus-usos ==="
```

```
check "GET /cupoes/meus-usos" "200" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/cupoes/meus-usos -H "Authorization: Bearer $TOKEN_CLIENTE")"
```

```
echo "==== [11] GET /carrinho ==="
```

```
check "GET /carrinho" "200" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/carrinho -H "Authorization: Bearer $TOKEN_CLIENTE")"
```

```
echo "==== [12] GET /perfis/me ==="
```

```
check "GET /perfis/me" "200" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/perfis/me -H "Authorization: Bearer $TOKEN_CLIENTE")"
```

```
echo "==== [13] GET /banners ==="
```

```
check "GET /banners" "200" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/banners)"
```

```
echo "==== [14] GET /deliveries?fornecedor_id=1 ==="
```

```
check "GET /deliveries?fornecedor_id=1" "200" "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/deliveries?fornecedor_id=1" -H "Authorization: Bearer $TOKEN_ADMIN")"
```

```
echo ""
```

```
echo "=====
```

```
echo " Resultado: $PASS passou / $FAIL falhou"
```

```
echo "=====
```

Resultado esperado: 14 passou / 0 falhou

✓ Resumo das Correções — v2.1.5

#	Endpoint	Problema Anterior	Correção
1	POST /requisicoes sem preco_sugerido	500	✓ String(?? 0) — bug resolvido após 3 rondas
2	GET /anexos/pessoa/{id}	500 (regressão)	✓ SQL puro
3	GET /pessoa-anexos/pessoa/{id}	500 (regressão)	✓ SQL puro
4	GET /banners	500	✓ SQL puro
5	PATCH /pessoas/:id	404	NEW Criado
6	PATCH /clientes/:id/limite-credito	404	NEW Criado
7	POST /contactos	404	NEW Criado
8	PATCH /contactos/:id	404	NEW Criado
9	DELETE /contactos/:id	404	NEW Criado
10	POST /pedidos/:id/chat	404	NEW Criado
11	Rota duplicada em AnexoController	—	✓ Removida

✓ Totais de Correções Acumuladas (v2.1.0 → v2.1.5)

#	Endpoint	Problema	Correção
1	GET /anexos/*	Requeria JWT	🌐 Público
2	POST /anexos/upload	id obrigatório	✓ Opcional
3	GET /anexos/produto/{id}	Sem url	✓ Fallback
4	GET /catalog/fornecedor/{id}/geolocalizacao	lat/lon trocados	✓ ST_Y/ST_X
5	POST /requisicoes sem preco_sugerido	500	✓ ?? 0
6	POST /requisicoes sem categoria_sugerida_id	500	✓ ?? 1
7	POST /avaliacoes pedido próprio	403	✓ Number()
8	GET /admin/usuarios/{id}	password_hash exposto	✓ Oculto
9	GET /cupoes/meus-usos	403	✓ Permissão + ordem de rotas
10	GET /carrinho	500 (FK constraint)	✓ idpessoa + SQL direto
11	GET /perfis/me	Não existia	NEW Criado
12	GET /contactos/pessoa/:id	Não existia	NEW Criado
13	GET /pedidos/:id/chat	Não existia	NEW Criado
14	GET /banners + CRUD	Não existia	NEW Criado (via pessoa_anexo)
15	GET /deliveries?fornecedor_id=X	Sem filtro	NEW Adicionado

#	Endpoint	Problema	Correção
16	config dos banners	Não persistia	 Coluna JSON em pessoa_anexo
17	JWT sem permissões	AuthService incompleto	 Corrigido
18	PATCH /pessoas/:id	Não existia	 Criado (v2.1.5)
19	PATCH /clientes/:id/limite-credito	Não existia	 Criado (v2.1.5)
20	POST/PATCH/DELETE /contactos	Não existia	 Criado (v2.1.5)
21	POST /pedidos/:id/chat	Não existia	 Criado (v2.1.5)

Última atualização: 14 de Setembro de 2026

Versão: 2.1.5

Equipa Backend — Tech Conectar

 contacto@techconectar.com

 <https://techconectar.com>